

# 供电设备图像异动检测系统

流程说明与模块拆解

2026 年 7 月 23 日

## 目录

|    |                            |    |
|----|----------------------------|----|
| 1  | 系统总览：一条图的两头                | 2  |
| 2  | 模块 1：数据准入——把文件读进来、数读对      | 2  |
| 3  | 模块 2：配准级联——把两张图对上          | 3  |
| 4  | 模块 3：部件映射门禁——对上之后，再确认”真能用” | 6  |
| 5  | 模块 4：差异检测——真·找不同           | 7  |
| 6  | 模块 5：证据产物——每一次判定都要能翻出来查    | 9  |
| 7  | 模块 6：服务层——把 1-5 串起来        | 10 |
| 8  | 模块 7：多部件聚合——一张图多个部件怎么办     | 11 |
| 9  | 双模式流程图对比                   | 12 |
| 10 | 完整数据流总结                    | 13 |
| 11 | 附录：关键指标解释                  | 13 |

## 1 系统总览：一条图的两头

系统的任务很简单：业务方拍了两张图——一张叫**标准源图**（这个设备正常时长什么样），一张叫**实时巡检图**（现在长什么样），另外在标准图上用 YOLO 框标出了要检查的部件。系统要做的事只有三件：

1. 先把两张图对上——实时图里的那个位置，到底对应标准图的哪个位置？
2. 对上了之后，把两个状态的同一片区域拿出来比——有没有哪里不一样？
3. 给出结论：没变化（高置信）、有变化（框出来）、还是说“我判断不了”（告知原因）。

整个流程是一条直线，分成**七个模块**，每个模块只做一件事，做完把结果交给下一个模块。如下图所示。

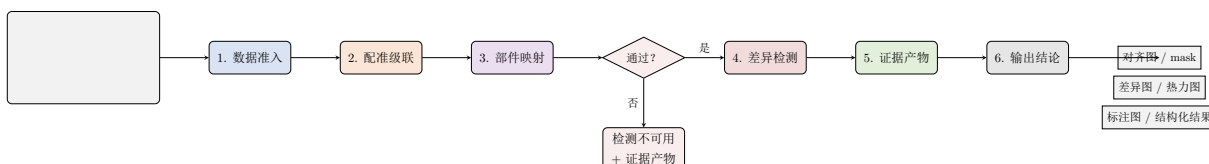


图 1: 系统主流程（单部件）

六个模块对应六个源文件，再加一个负责“多个部件凑一张图”的聚合层（模块 7），构成了整个系统。下面按顺序逐个拆解。

## 2 模块 1：数据准入——把文件读进来、数读对

### 对应文件

- `references.py` —— 案例目录发现、图片读取
- `roi.py` —— YOLO 坐标文件解析

### 做了什么事

1. 扫描一个案例目录，找到三样东西：

- 对比截图.jpg —— 实时巡检拍的那张
- 标准源图.jpg —— 这个机位的标准参照
- 标准源图坐标.txt —— YOLO 框文件，每行一个部件：

类别 ID      中心 x      中心 y      宽度      高度

坐标都归一化到  $[0, 1]$ （即 0.5 表示画面正中间，1.0 表示最右/最下）

- 可选：新增标准源图 0.jpg, 新增标准源图 1.jpg, ... ——其他时刻拍的标准图（比如晚上拍的）
- 2. 图片读取走的是 `numpy.fromfile + cv2.imdecode` 路径，而不是直接用 OpenCV 的 `imread`——因为 Windows 中文路径 OpenCV 不认
- 3. YOLO 坐标文件解析时做了容错处理：坐标从  $[-0.01, 1.01]$  都在容忍范围内（稍微出框一点可以裁回来），超出这个范围才算格式非法
- 4. 解析出来的每行坐标转成像素矩形： $x = (cx - w/2) \times W$ ,  $y = (cy - h/2) \times H$ ,  $w = r_w \times W$ ,  $h = r_h \times H$ （其中  $W, H$  是标准图的宽高， $r_w, r_h$  是归一化的宽高）

### 输入 → 输出

| 输入            | 输出               | 说明                                  |
|---------------|------------------|-------------------------------------|
| 案例目录路径        | CaseInput 对象     | 包含实时图路径、标准图路径列表、YOLO 文件路径           |
| YOLO 坐标文件     | list[Roi]        | 每行解析成一个 Roi 对象（类别、中心 x/y、宽、高，全归一化）  |
| 图片路径          | np.ndarray (BGR) | 三维数组， $H \times W \times 3$ , uint8 |
| 归一化 Roi + 图尺寸 | (x, y, w, h)     | 像素坐标的矩形，整数                          |

## 3 模块 2：配准级联——把两张图对上

### 对应文件

- `alignment.py` ——完整的对齐流水线 + 三态诊断分类
- `config.py` 中 `AlignmentThresholds` 部分——所有门槛都来自配置，不写死

### 做了什么事

配准级联是一条“先快后稳”的串联流水线。它先用 ORB 完成一次完整检查；只有 ORB 给出不可靠或不可用、且配置明确允许时，才让 SuperPoint+LightGlue 补一次全图匹配点。补点不是放行：补出的点仍要重新走 RANSAC、投影几何、有效重叠和 ECC 微调这几道原门槛。默认配置关闭这条补路，待依赖许可审批完成后才可在日常检测中打开。

#### 第 1 步：ORB 特征提取

两张灰度图各提 2000 个 ORB 特征点（Oriented FAST + Rotated BRIEF，一种经典的、不需要 GPU 的特征检测算法）。每个点附着一个 256 位的二进制描述符。

## 第 2 步：特征匹配

用暴力匹配器（BFMatcher）把标准图的描述符跟实时图的比，选”最近邻距离除以次近邻距离  $< 0.75$ ”的点对留下来——比值越小，匹配越可靠。

## 第 3 步：RANSAC 求单应矩阵

拿匹配点的坐标对，用 RANSAC（随机抽样一致）算法估计一个  $3 \times 3$  的单应矩阵  $H$ 。它的作用是把标准图上的坐标映射到实时图上： $[x_{\text{live}}, y_{\text{live}}, 1]^T \sim H \cdot [x_{\text{std}}, y_{\text{std}}, 1]^T$ 。RANSAC 会同时把”跟大多数点不一致的”点标为外点，只留内点。

这一步算出来的几个关键数字：内点数量、内点率（内点/匹配点）、重投影误差（内点被  $H$  映射后的平均像素偏差）、空间覆盖度（内点占标准图面积的比例）。

## 第 4 步：算投影几何

把标准图的四个角通过  $H$  投到实时图里，算：中心位移（两个图的画面中心差了多少像素）、角点位移（四个角各差多少）、投影面积比（投过去的面积占实时图的比例）、有几个角落在实时图画面内。

## 第 5 步：Warp + 有效 mask

把实时图按照  $H^{-1}$ （反变换）扭曲到标准图的坐标系里：标准图每个像素去找实时图上对应的位置，用双线性插值填颜色。扭曲之后会有黑边（实时图够不到的位置），这些地方在”有效 mask”里标为无效。

## 第 6 步：ECC 微调

warp 之后，两图可能还有几个像素的对不齐。用 OpenCV 的 ECC（增强相关系数）算法在局部做一个仿射微调——在有效 mask 范围内，找一个能让两图灰度值更一致的小幅平移 + 旋转 + 缩放。收敛了就得到更高的对齐精度；如果 50 次迭代还不收敛，记录”ECC 未收敛”并把这个事实当作负证据传给后面的门禁。

## 第 7 步：近恒等 + 父子图检测

如果  $H$  近乎恒等变换（中心位移不到画面对角线的 5%，面积比在  $[0.85, 1.15]$  之内）——说明两张图的拍摄位置基本没动。但是，如果同一个  $H$  下内点覆盖度低于 0.2 且 ECC 还没收敛——这就是典型的”父子图”场景：标准图是局部放大，实时图是广角，虽然  $H \approx I$  看起来不错（因为特征点凑巧在地面纹理上匹配上了），但实际上两图的尺度完全对不上。这种情况要专门打上标记（suspected\_zoom\_parent\_child\_mismatch），后面的门禁会据此拦下。

## 第 8 步：三态诊断分类

综合前面 1-7 步的所有指标，给出三态结论：

- **usable（可用）**：所有指标都在配置的门槛之上，可以继续往下走
- **unreliable（不可靠）**：虽然还能算出一个变换，但有关键证据不达标。日常检测到这里就停止，统一输出”检测不可用”；后面即使为了排查而算出了部件指标，也不能把它变成”无异动”、“部件差异”或”图像不匹配”

- **unavailable (不可用)**: 根本配不出变换, 或者投影几何非法——直接终止, 报”检测不可用”

### ORB 失败时的辅助尝试

SuperPoint 是一种从图像中找稳定特征点的模型, LightGlue 是一种把两张图的特征点配成对的模型。它们只回答”这两个像素位置可能对应”, 不回答”两张图已经可信对齐”, 更不会直接给部件画框。模型输出的匹配点会像 ORB 点一样重新交给第 3-8 步处理; 任何一项门槛不过, 结果仍是”检测不可用”。同一对标准图和实时图在一个多部件案例中只计算一次对齐结果, 后续部件复用该结果, 避免重复推理。

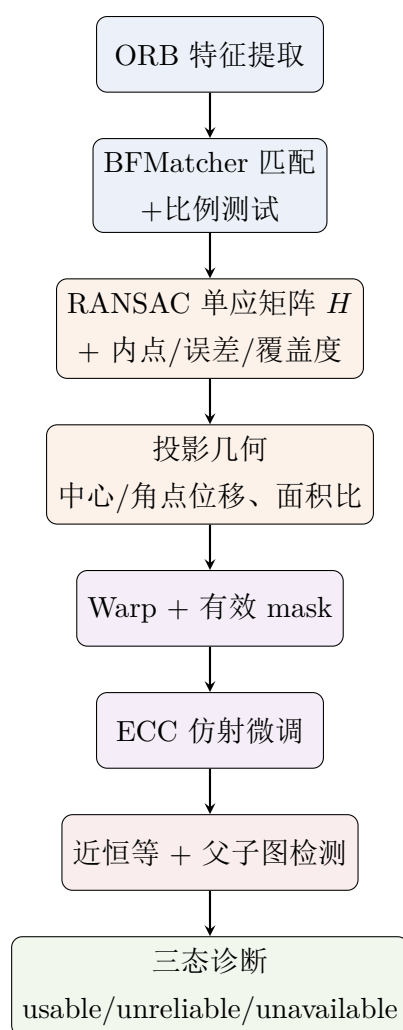


图 2: 配准级联的主流程; ORB 不可靠时, 可选的 SuperPoint+LightGlue 只补匹配点, 随后仍回到第 3-8 步

## 输入 → 输出

| 输入                                   | 输出              | 说明                                                                    |
|--------------------------------------|-----------------|-----------------------------------------------------------------------|
| 标准灰度图、<br>实时灰度图、<br>CalibratedConfig | AlignmentResult | 含 $H$ 矩阵、 $H^{-1}$ 、<br>aligned_live 图、<br>valid_mask、全部配准<br>指标、诊断分类 |

## 4 模块 3：部件映射门禁——对上之后，再确认”真能用”

## 对应文件

- mapping.py ——四道组件级检查

## 做了什么事

配准给出的是一个”全图”级别的对齐关系。但系统要比较的是某一个具体部件的 ROI。所以需要再走一遍跟部件直接相关的检查——看看这个部件在实时图里到底能不能拿出来比。

检查顺序是固定的，前一道不过就不走最后一道（短路）：

## 第 1 道：候选定位（B 层）

把标准图上的 ROI（要比较的那一小块区域）四个角通过  $H$  投射到实时图里，看投过去的四边形有多少在画面内。当前正式配置要求至少 70% 在画面内。达不到说明这个位置已经不能可靠对齐，报”配准失败”。

## 第 2 道：有效像素重叠（B 层）

在 ROI 区域内，有效 mask 覆盖了多少像素？当前正式配置要求至少 85%。达不到说明要么是扭曲后的黑边占了太大面积，要么是配准不稳——报”配准失败”。

## 第 3 道：有效分辨率（C 层）

根据  $H$  在部件附近的缩放关系，算出：实时图上这个部件到底占了多少个像素？当前正式配置要求有效采样尺度至少 0.4 倍（effective\_resolution\_scale\_min），有效宽和高都至少 48 像素。达不到说明实时图里的部件太小或太糊，放大图片也不会凭空长出细节，报”匹配不确定”。

## 第 4 道：配准后表现一致性（D 层）

这是最关键的一道。在 ROI 里用有效 mask 内那些”配得上”的像素，算两个数：

- NCC（归一化互相关）：

$$\text{NCC} = \frac{\sum (A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum (A_i - \bar{A})^2} \cdot \sqrt{\sum (B_i - \bar{B})^2}}$$

范围  $[-1, 1]$ 。1 表示两张图的灰度”变化走势”完全一致；0 表示互不相关；负数表示方向反了（标准图暗的地方实时图反而亮）

- **SSIM（结构相似性）**：在每个  $11 \times 11$  的小窗口里同时看平均亮度、明暗反差和纹理是否同向，再把所有窗口的结果取平均：

$$\text{SSIM}(A, B) = \frac{(2\mu_A\mu_B + C_1)(2\sigma_{AB} + C_2)}{(\mu_A^2 + \mu_B^2 + C_1)(\sigma_A^2 + \sigma_B^2 + C_2)}$$

其中  $\mu$  是平均亮度， $\sigma$  是明暗变化程度， $\sigma_{AB}$  表示两张图是否一起变亮或变暗； $C_1, C_2$  是防止分母为零的小常数。范围大致在  $[-1, 1]$ ，越接近 1 越像。

原始 NCC 不到 0.35，或原始 SSIM 不到 0.2，只说明”这两块看起来不像”，可能是机位拍错，也可能只是日夜、曝光或阴影不同。这时先不进入差异检测，而是做三次复核：把两图的亮度分布拉到相近后再算 NCC；去掉局部明暗起伏后再算 NCC；只看边缘强弱后再算 NCC。

**图像不匹配的确认规则**：全图配准和前三道部件检查必须全部通过；原始 NCC 也必须低于 0.35；并且上面三次复核的 NCC 都仍低于 0.35。只有同时满足这些条件，才输出”图像不匹配异动”。如果任一次复核还能对上，或者只有 SSIM 低，则证据还不够，输出”检测不可用”，绝不往下走比较。

**还有一个额外逻辑**：映射门禁同时算了一个 `comparison_scale`（比较用的缩放倍率），取  $\min(1.0, S)$ （ $S$  是有效分辨率尺度）——如果实时图比标准图糙（分辨率低），就把两图都缩小到实时图的分辨率再比，不靠上采样造假。

## 输入 → 输出

| 输入                                                  | 输出                                    | 说明                                                                 |
|-----------------------------------------------------|---------------------------------------|--------------------------------------------------------------------|
| 标准 BGR 图、warp 后的实时图、有效 mask、 $H$ 矩阵、ROI 矩形、实时图尺寸、配置 | <code>ComponentMappingEvidence</code> | 含四道结果、 <code>failure_reason</code> 、 <code>comparison_scale</code> |

## 5 模块 4：差异检测——真·找不同

### 对应文件

- `compare.py` ——抗光照颜色差、局部结构差、图像清晰度检查和候选分级

### 做了什么事

模块 3 全部通过之后，两图已经对齐好了，ROI 内的像素才允许进入”找不同”。这里不再直接拿原图逐像素相减，而是分别看颜色和结构，并先判断实时图是不是已经糊到不能可靠比较。

### 通道 1: 先压掉光照影响, 再看颜色

BGR 先转 Lab 颜色空间:  $L$  表示明暗,  $a$ 、 $b$  表示两条颜色轴。先在有效 ROI 内把实时图的  $L$  亮度分布拉到与标准图相近, 再计算颜色距离:

$$\Delta E = \sqrt{(L_1 - L_2)^2 + (a_1 - a_2)^2 + (b_1 - b_2)^2}$$

这里的  $\Delta E$  就是两种颜色在 Lab 空间里的直线距离。全图亮度中位数差得越多, 颜色通道的权重越低, 但不会低于配置下限。这样日夜、曝光和阴影不会轻易变成颜色异动; 局部的油污、变色或涂层变化仍会留下信号。

### 通道 2: 看局部结构是否变了

两图都先减去周围一大片区域的平均亮度, 去掉缓慢变化的阴影; 再用 Sobel 边缘算子分别求水平和竖直方向的边缘。Sobel 不是缩写, 它是一种用周围  $3 \times 3$  像素估计边缘方向的算法。结构分数主要看两张图边缘方向是否一致:

$$s_{\text{structure}} = \frac{1 - \cos \theta}{2} \times \min \left( 1, \frac{\max(G_1, G_2)}{T_G} \right)$$

其中  $\theta$  是两张图在同一像素处的边缘方向夹角,  $G_1, G_2$  是边缘强度,  $T_G$  是配置中的强边缘尺度。方向一致时分数接近 0; 一个边缘消失、新增或方向明显改变时分数升高。这样比单纯比较边缘强弱更不容易把曝光变化误当成裂纹或缺件。

### 先检查清晰度

在出业务结论前, 系统用 Laplacian (二阶边缘变化) 计算标准图和实时图在有效 ROI 内的清晰度, 再算较小值与较大值的比:

$$q = \frac{\min(S_{\text{std}}, S_{\text{live}})}{\max(S_{\text{std}}, S_{\text{live}})}$$

当  $q$  低于配置下限时, 说明实时图相对标准图明显变糊。此时仍保留差异候选和五张证据图, 但日常检测统一返回“检测不可用”, 不会把拍糊了误报成设备异动, 也不会假装“无异动”。

### 合并与候选生成

两条通道的分数图取逐像素最大值, 得到一张“融合差异热力图”。在这张热力图上:

1. 大于判定门槛 (开发值 = 0.5) 的位置标为 1, 其余标为 0
2. 形态学操作 (开运算) 去噪
3. 找轮廓, 把每个连通区域变成一个候选框  $(x, y, w, h)$
4. 算每个候选的面积和置信度。置信度由区域内热力图平均值和面积共同得到; 小区域不会因一个亮点就拿到很高把握
5. 所有候选都保留为原始证据。面积过小或置信度过低的候选会标成“仅供复核”, 画成橙色框; 只有同时达到面积和置信度门槛的候选才允许触发业务异动, 画成红色框



## 输入 → 输出

| 输入                                         | 输出            | 说明                                         |
|--------------------------------------------|---------------|--------------------------------------------|
| 标准 BGR、warp 后实时 BGR、有效 mask、ROI 矩形、配置、比较倍率 | CompareResult | 含二值差异 mask、float32 热力图、清晰度证据、全部候选和可触发结论的候选 |

## 6 模块 5：证据产物——每一次判定都要能翻出来查

### 对应文件

- `artifacts.py` —— 五类证据图的生成

### 做了什么事

不管最终结论是”无异动””有异动”还是”检测不可用”，每次都产出同样的五张图，存在调用方指定的目录里。这样以后有人想质疑”为什么判成这样”，可以直接翻证据。

1. **alignment.png**（对齐证据图）——左半边是标准图，右半边是实时图，中间用绿色线连的是标准图四角投影到实时图的位置，黄色线连的是 RANSAC 内点对（最多 120 根，间隔采样保证看得清）
2. **valid\_mask.png**（有效区域图）——白色表示这块像素可以比，黑色表示 warp 黑边或者无效区。二值图
3. **difference\_mask.png**（差异区域图）——白色表示被判定为”有差异”的像素。在整图大小上画，方便跟原图对着看
4. **difference\_heatmap.png**（差异热力图）——彩色热力图，越红表示差异分数越高。也是整图大小
5. **annotated.png**（标注图）——在实时图上画框：绿色框是 ROI 投影过去的位置；红色框是可触发结论的候选，橙色框是仅供复核的原始候选；候选框旁写该区域的置信度。图中没有颜色条

如果流程在中途就被拦下（比如配安装不上、映射没过），照样生成对齐图和有效区域图——只是差异图是空的。这样”为什么没比出来”也是可以验证的，不会出现”没结果”但”不知道是不是根本没跑”的情况。

## 输入 → 输出

| 输入                                 | 输出           | 说明                   |
|------------------------------------|--------------|----------------------|
| 标准图、实时图、 $H$ 矩阵、配准结果、差异结果、候选框、输出目录 | 5 个 PNG 文件路径 | ArtifactSet，每个都是本地路径 |

## 7 模块 6：服务层——把 1-5 串起来

### 对应文件

- service.py —— ImageComparisonService，单部件对比的唯一入口

### 做了什么事

service 就是”调度员”。它负责把模块 1-5 按正确的顺序和分支逻辑串起来：

1. 读图、读 ROI（调模块 1）
2. 跑配准（调模块 2）
3. 如果全图配准诊断不是 usable → 写不可用证据，返回”检测不可用”。为了排查问题可以保留已算出的部件指标，但它们不能进入业务判断
4. 全图配准可用时，才把实时图扭到标准图坐标，并跑部件映射（调模块 3）
5. 映射确认”图像不匹配” → 写证据，返回”图像不匹配异动”
6. 其他映射不通过 → 写不可用证据，返回”检测不可用”（带原因）
7. 跑差异检测（调模块 4）
8. 把候选区域从标准图坐标系反投回实时图（用  $H$ ）
9. 写证据产物（调模块 5）
10. 若清晰度不足 → ”检测不可用”；否则有可触发结论的候选 → ”检出异动”，只有没有这类候选时才是”高置信无异动”。原始小候选仍会保留在结果和证据图里，但不会单独触发异动

#### 两种调用方式：

- compare() —— 日常检测模式：必须传版本化的标定配置，返回带业务结论的 ComparisonResult
- observe() —— 标定观察模式：可用开发配置，返回 CalibrationComponentObservation，不含业务结论（没有 normal/anomaly 字段）

## 输入 → 输出

| 输入                                                    | 输出                                                   | 说明                        |
|-------------------------------------------------------|------------------------------------------------------|---------------------------|
| 标准图路径、实时图路径、YOLO 字符串 ("17 0.5 0.5 0.4 0.5")、输出目录、配置路径 | ComparisonResult (日常) / CalibrationComponentUse (标定) | 日常含四类结论；标定只含观察记录和全部证据产物路径 |

## 8 模块 7：多部件聚合——一张图多个部件怎么办

### 对应文件

- multi\_component.py ——MultiComponentImageComparisonService

### 做了什么事

一张标准图上的 YOLO 文件里有多个部件，系统要先选一个”最好对齐”的参考图（可能有多张标准图），然后对每个部件挨个跑模块 6。

#### 选参考图的逻辑：

1. 对每张参考图（主标准图 + 可选的新增标准图 0/1/2...），跟实时图跑一次配准
2. 给每张参考图打分：usable 优先 > unreliable > unavailable；同档内比内点率、重投影误差、有效重叠
3. 选分数最高的那张作为”选中参考图”
4. 所有部件都用同一张参考图——因为后续的整图结论需要一个统一的坐标系

#### 聚合结论的逻辑：

- 任一部件”图像不匹配异动” → 整图”图像不匹配异动”
- 否则，任一部件”部件差异异动” → 整图”部件差异异动”
- 没有异动、但任一部件”检测不可用” → 整图”检测不可用”
- 所有部件全部”高置信无异动” → 整图”高置信无异动”

输入 → 输出

| 输入   | 输出                                                       | 说明                                      |
|------|----------------------------------------------------------|-----------------------------------------|
| 案例目录 | CalibrationObservation (标定) / ImageComparisonResult (日常) | 选中中的参考图、每个部件的结论/观察、整图聚合结论、manifest JSON |

9 双模式流程图对比

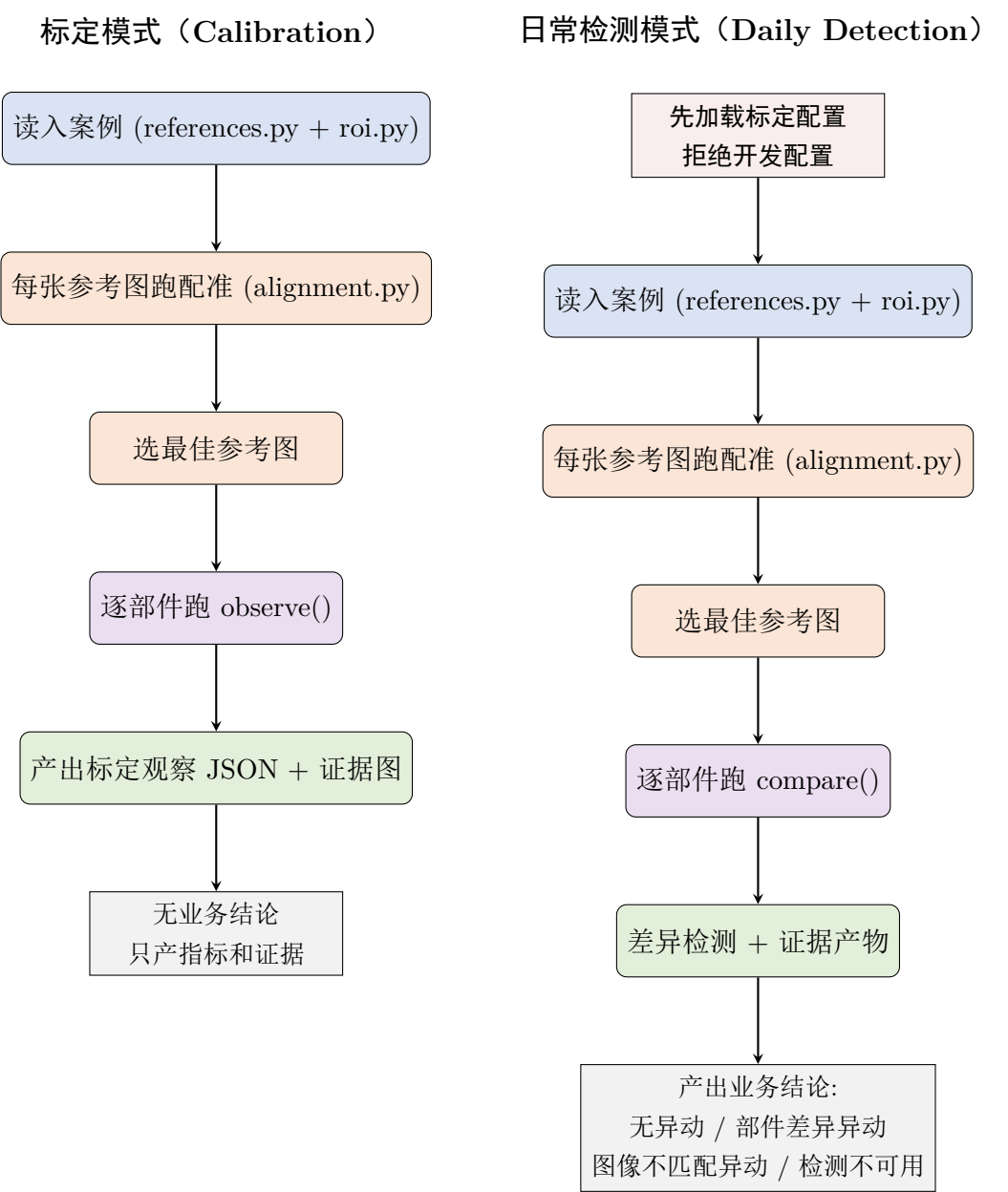


图 3: 标定模式（左）和日常检测模式（右）的流程对比

## 10 完整数据流总结

### 11 附录：关键指标解释

- **内点率 (inlier\_rate)**: RANSAC 给出的内点数  $\div$  特征匹配数。越高说明“一致的特征点越多”，配准越可信
- **重投影误差 (reprojection\_error)**: 内点被  $H$  映射后，跟目标位置的偏差（像素）。越低越好
- **空间覆盖度 (spatial\_coverage)**: 内点在标准图上的分布面积占比。如果所有内点挤在一个角落，就算内点率很高也可能配错
- **有效重叠率 (valid\_overlap\_ratio)**: warp 后有效像素占标准图的比例。黑边太多说明旋转/缩放太大
- **ECC 相关性 (ecc\_correlation)**: 微调后两图的全局相关程度。越高说明微调更成功
- **有效分辨率尺度 (effective\_resolution\_scale)**:  $H$  的雅可比矩阵最小奇异值——实时图上一个像素相当于标准图上多少个像素。 $< 1$  说明实时图更糙
- **比较倍率 (comparison\_scale)**:  $\min(1.0, S)$ ，其中  $S$  是有效分辨率尺度，确保比较时不用上采样
- **标定模式 vs 日常检测模式**: 标定模式可用开发配置，只产观察数据和证据图，不产业务结论。日常检测模式必须在入口加载带版本号的标定配置，产四类业务结论：高置信无异动、部件差异异动、图像不匹配异动、检测不可用。这是整个系统的“安全闸”——防止用占位阈值冒充生产判定

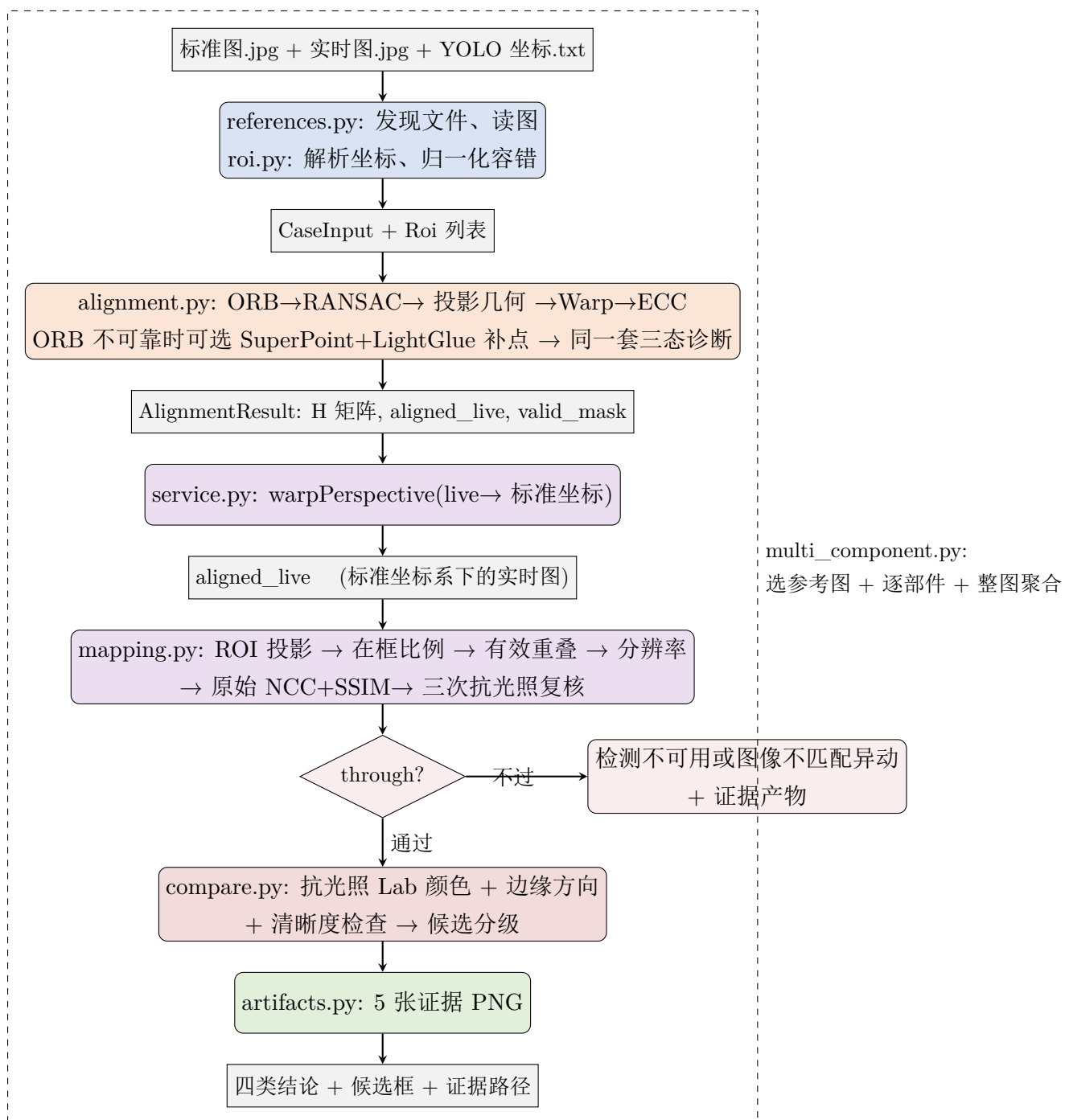


图 4: 完整数据流（从上到下，每个箭头是一层产出）